Cours

Bases de données

Cours 4
Le Langage SQL (Partie 1)

Création, insertion et mise à jour des données

- Niveau: B1
- Année universitaire: 2025-2026

Dr. Salma Sassi

salma.sassi@ece.fr

SQL

Structured Query Language (langage structuré de requêtes) est le langage standard pour créer, manipuler et interroger les bases de données relationnelles.

3 catégories : DDL, DML et DCL.

- **Langage de Définition de Données (DDL) :** définir le schéma de la base de données
- **Langage de Manipulation de Données (DML) :** interroger et modifier les données de la base
- **Langage de contrôle d'accès aux données**

SQL/Types

	Type	Description	Taille
Types numériques	INTEGER	Type des entiers relatifs	4 octets
	SMALLINT	Idem.	2 octets
	BIGINT	Idem.	8 octets
	FLOAT	Flottants simple précision	4 octets
	DOUBLE PRECISION	Flottants double précision	8 octets
	REAL	Synonyme de FLOAT	4 octets
	NUMERIC (M, D)	Numérique avec précision fixe.	M octets
	DECIMAL (M, D)	Idem.	M octets
Chaînes de caractères	CHAR(M)	Chaînes de longueur fixe	M octets
	VARCHAR(M)	Chaînes de longueur variable	$L+1$ avec $L \leqslant M$
	BIT VARYING	Chaînes d'octets	Longueur de la chaîne.
Types de date et heure	DATE	Date (jour, mois, an)	env. 4 octets
	TIME	Horaire (heure, minutes, secondes)	env. 4 octets
	DATETIME	Date et heure	8 octets
	YEAR	Année	2 octets

Création des tables

- Une table
  - est une unité de stockage élémentaire, composée de lignes et de colonnes
  - Le nom d'une table
    - Doit commencer par une lettre
    - Ne doit pas dépasser les 30 caractères
    - Ne peut contenir que les caractères A à Z, a à z, 0 à 9, _, $, et #
    - Ne doit pas porter le nom d'un autre objet appartenant au même utilisateur
    - Ne doit pas être un mot réservé

Création des tables

Syntaxe

CREATE TABLE *nom de la table* (

attribut1 type(attribut1), attribut2 type (attribut2) …

);

Exemple

```
CREATE TABLE Internaute (
        email VARCHAR (50),
        nom VARCHAR (20),
        prenom VARCHAR (20),
        motDePasse VARCHAR (60),
        anneeNaiss INTEGER (4)
    );
```

Contraintes

- **Contraintes qu'on peut demander:**
  - Un attribut doit toujours avoir une valeur (NOT NULL)
  - Un attribut (ou ensemble d'attributs) constitue(nt) la clé de la relation
  - Un attribut dans une table est lié à la clé primaire d'une autre table
  - La valeur d'un attribut doit être unique au sein de la relation
  - D'autres types de règles…

Contraintes

- Syntaxe

```
[CONSTRAINT nom_c ]
{ [NOT] NULL | UNIQUE | PRIMARY KEY | FOREIGN KEY | REFERENCES
[Schéma].table(colonne) [ON DELETE CASCADE] | CHECK (condition)}]
```

- Exemple

```
CREATE TABLE EMP (
        empno NUMBER(4),
        ename VARCHAR2(10),
        …
        deptno NUMBER(7,2) NOT NULL,
        CONSTRAINT emp_emmno_pk PRIMARY KEY (EMPNO)
);
```

Contraintes

NULL / NOT NULL

- Il y a toujours une valeur!

```
CREATE TABLE Internaute
(email VARCHAR (50) NOT NULL,
nom VARCHAR (20) NOT NULL,
prenom VARCHAR (20),
motDePasse VARCHAR (60) NOT NULL,
anneeNaiss INTEGER (4))
```

Contraintes

NULL / NOT NULL

- Absence de valeur (pas valeur zéro ou chaîne vide!!!)
  - Pas d'opération incluant le NULL
  - Pas de comparaison avec un NULL

Comment forcer un attribut?

- **DEFAULT**

```
CREATE TABLE Cinéma
(nom VARCHAR (50) NOT NULL,
adresse VARCHAR (50) DEFAULT 'Inconnue')
```

Contraintes

Clé primaire (Primary key)

- **Défini :**

Au niveau table

```
CREATE TABLE EMP (
        empno NUMBER(4),
        ename VARCHAR2(10),
        …
        deptno NUMBER(7,2) NOT NULL
        CONSTRAINT emp_emmno_pk PRIMARY KEY (empno)
);
```

Nom de la contrainte

Au niveau colonne

```
CREATE TABLE EMP (
        empno NUMBER(4) PRIMARY KEY,
        ename VARCHAR2(10),
        …
        deptno NUMBER(7,2) NOT NULL
        );
```

Contraintes

Clé primaire (Primary key) → **Clé primaire composée**

```
CREATE TABLE Notation
    (idFilm INTEGER NOT NULL,
    email VARCHAR (50) NOT NULL,
    note INTEGER DEFAULT 0
    PRIMARY KEY (idFilm, email));
```

Contraintes

UNIQUE

- Défini au niveau table ou colonne

Au niveau table

```
SQL> CREATE TABLE EMP (
        empno NUMBER(4) PRIMARY KEY,
        ename VARCHAR2(10),
        …
        deptno NUMBER(7,2) NOT NULL,
        CONSTRAINT emp_nom_KT UNIQUE(ename)
);
```

Au niveau Colonne

```
SQL> CREATE TABLE EMP (
        empno NUMBER(4) PRIMARY KEY,
        ename VARCHAR2(10) UNIQUE,
        …
        deptno NUMBER(7,2) NOT NULL,
);
```

Contraintes

UNIQUE

```
CREATE TABLE Artiste
        (id INTEGER NOT NULL,
        nom VARCHAR (30) NOT NULL,
        prenom VARCHAR (30) NOT NULL,
        anneeNaiss INTEGER,
        PRIMARY KEY (id)
        UNIQUE (nom, prenom));
```

Contraintes

FOREIGN KEY

- Défini au niveau table ou colonne
- Associé à
  - REFERENCES pour identifier la table et la colonne de la table maître
  - ON DELETE CASCADE pour autoriser la suppression d'une ligne dans la table maître et des lignes dépendantes dans la table détail

Contraintes

FOREIGN KEY

Nom explicite de la contrainte

Référence à la table d'origine de la clé

Nom de la clé étrangère

Nom de la clé primaire dans sa table d'origine

```
SQL> CREATE TABLE EMP (
        empno NUMBER(4) PRIMARY KEY,
        ename VARCHAR2(10),
        …
        deptno NUMBER(7,2) NOT NULL,
        CONSTRAINT emp_deptno_KT FOREIGN KEY (deptno) REFERENCES DEPT (deptno));
```

Au niveau Table

```
CREATE TABLE Film
    (idFilm INTEGER NOT NULL, titre VARCHAR (50) NOT NULL, annee INTEGER NOT
    NULL, idRealisateur INTEGER,
        codePays INTEGER, P
        PRIMARY KEY (idFilm),
        FOREIGN KEY (idRealisateur) REFERENCES Artiste
        FOREIGN KEY (codePays) REFERENCES Pays);
```

Contraintes

FOREIGN KEY

Au niveau Colonne

```
SQL> CREATE TABLE EMP (
       empno NUMBER(4) PRIMARY KEY,
       ename VARCHAR2(10),
       …
       deptno NUMBER(7,2) NOT NULL FOREIGN KEY REFERENCES DEPT (deptno));
```

Clé étrangère: effets et contraintes d'intégrité

- Une **clé étrangère (FOREIGN KEY)** garantit la **cohérence** entre deux tables liées. Mais certaines opérations peuvent **violer cette contrainte** 👇

⚠️ **Exemples de violations possibles**

Opération	Problème rencontré	Effet
• Insertion dans Film avec un idRealisateur **inexistant** dans Artiste	Le SGBD ne trouve pas la référence	❌ Rejet de l'insertion
• Suppression d'un Artiste qui a encore des films associés	Films "orphelins"	❌ Rejet de la suppression

```
CREATE TABLE Film
  (idFilm INTEGER NOT NULL,
  titre VARCHAR (50) NOT NULL,
  annee INTEGER NOT NULL,
  idMES INTEGER,
  codePays INTEGER,
  PRIMARY KEY (idFilm),

  FOREIGN KEY (idRealisateur)
  REFERENCES Artiste,
  FOREIGN KEY (codePays)
  REFERENCES Pays);
```

Contraintes

FOREIGN KEY

La destruction d'un Réalisateur doit déclencher la mise à NULL (**SET NULL**) de la clé étrangère *idRealisateur* pour tous les films qu'il a réalisés :

```
CREATE TABLE Film
        (titre VARCHAR (50) NOT NULL,
        annee INTEGER NOT NULL,
        idMES INTEGER,
        codePays INTEGER,
        PRIMARY KEY (titre),
        FOREIGN KEY (idRealisateur) REFERENCES Artiste ON DELETE SET NULL
        FOREIGN KEY (codePays) REFERENCES Pays);
```

Contraintes

FOREIGN KEY

- **Quand on détruit un cinéma, on veut également détruire les salles ;**
- **Quand on modifie la clé d'un cinéma, on veut répercuter la modification sur ses salles**
  - **CASCADE** permet d'appliquer la même opération

```
CREATE TABLE Salle
        (nomCinema VARCHAR (30) NOT NULL,
        no INTEGER NOT NULL,
        capacite INTEGER,
        PRIMAR KEY (nomCinema, no),
        FOREIGN KEY (nomCinema) REFERENCES Cinema
                ON DELETE CASCADE
                ON UPDATE CASCADE
       );
```

Contraintes

FOREIGN KEY

Action	Effet sur la table fille (Film)	Exemple SQL
ON DELETE SET NULL	Remplace la valeur par NULL si le réalisateur est supprimé	FOREIGN KEY (idRéalisateur) REFERENCES Réalisateur(idRéalisateur) ON DELETE SET NULL
ON DELETE CASCADE	Supprime aussi les films associés	FOREIGN KEY (idRéalisateur) REFERENCES Réalisateur(idRéalisateur) ON DELETE CASCADE
ON DELETE RESTRICT *(par défaut)*	Interdit la suppression du réalisateur s'il est encore lié à un film	FOREIGN KEY (idRéalisateur) REFERENCES Réalisateur(idRéalisateur) ON DELETE RESTRICT

Contraintes

CHECK

- La contrainte **CHECK** permet de **vérifier qu'une valeur insérée ou modifiée respecte une condition logique**.
- Définie au niveau table ou colonne

Au niveau table

```
CREATE TABLE EMP (
      empno NUMBER(4) PRIMARY KEY,
      ename VARCHAR2(10),
      …
      deptno NUMBER(7,2) NOT NULL,
      CONSTRAINT emp_deptno_KT CHECK (deptno BETWEEN 10 AND 99), …));
```

Contraintes

CHECK

Au niveau Colonne

```
CREATE TABLE Film
        (titre VARCHAR (50) NOT NULL,
        annee INTEGER CHECK (annee BETWEEN 1890 AND 2000) NOT NULL,
        genre VARCHAR (10) CHECK (genre IN ('Histoire','Western','Drame')),
        idMES INTEGER,
        codePays INTEGER,
        PRIMARY KEY (titre),
        FOREIGN KEY (idMES) REFERENCES Artiste
        FOREIGN KEY (codePays) REFERENCES Pays);
```

Contraintes

Suppression de contrainte

```
ALTER TABLE nom_table
DROP PRIMARY KEY | UNIQUE (colonne) |
      CONSTRAINT nom_contrainte [CASCADE]
);
```

Ex: Supprimer la contrainte PRIMARY KEY de la table DEPT, ainsi que la contrainte FOREIGN KEY définie sur la colonne DEPTNO de la table EMP

```
ALTER TABLE DEPT (
      DROP PRIMARY KEY CASCADE
);
```

Contraintes

Désactivation/Activation d'une contrainte

```
ALTER TABLE nom_table
[DISABLE | ENABLE] CONSTRAINT nom_contrainte [CASCADE]
);
```

Exemple

```
ALTER TABLE Employe
DISABLE CONSTRAINT emp_deptno_FK;
```

```
ALTER TABLE Employe
ENABLE CONSTRAINT emp_deptno_FK;
```

Contraintes

Comment afficher les contraintes d'une table?

```
SELECT constraint_name, constraint_type, search_condition
      FROM user_constraints
      WHERE table_name = 'EMP'
);
```

Exemple

Constraint_Name	Constraint-type	Search_Condition
SYS_C003042	**C** (Signifie CHECK)	EMPNO is not Null
SYS_C003044	**U** (Signifie Unique)	
TEST_TT	**P** (Signifie Primary Key)	

ALTER TABLE: modification du schéma

- Permet de modifier la structure d'une table (Ajout d'une colonne, modification, définition d'une valeur par défaut, supprimer une colonne, etc.)

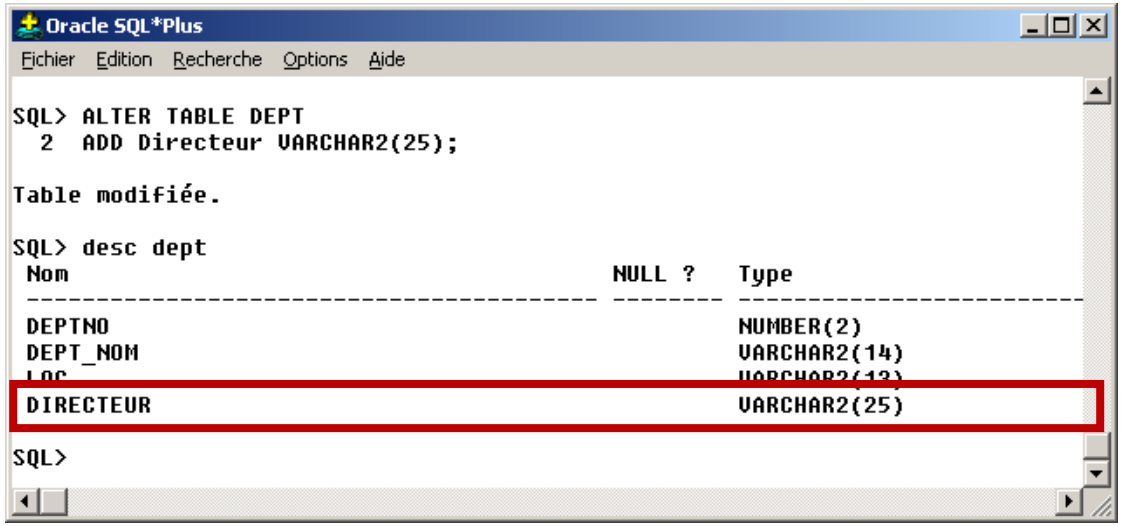
- La modification d'une valeur par défaut ne s'appliquer qu'aux insertions ultérieures dans la table

ALTER TABLE: modification du schéma

- ALTER TABLE
  - Syntaxe

```
SQL> ALTER TABLE nom_table
[ADD|MODIFY|DROP|RENAME COLUMN] (colonne1 datatype [DEFAULT expr] [TO
nouvelle_colonne] [CASCADE]
[, colonne2 …);
```

- **Ex**:



ALTER TABLE: modification du schéma

Action :

- *ADD: ajouter*
- *MODIFY: modifier*
- *DROP: supprimer*
- *RENAME: renommer*

ALTER TABLE: modification du schéma

Action : ADD

ALTER TABLE nom_table **ADD COLUMN** nom_colonne **type**

- ALTER TABLE Film
  ADD **COLUMN** region VARCHAR(10);

Le mot-clé **COLUMN** est **optionnel**

ALTER TABLE: modification du schéma

Action : MODIFY

ALTER TABLE nom_table **Modify COLUMN** nom_colonne **type**

- ALTER TABLE Film
  MODIFY region VARCHAR(30) NOT NULL;

ALTER TABLE: modification du schéma

Action : DROP

ALTER TABLE nom_table **DROP COLUMN** nom_colonne

- ALTER TABLE Film
  DROP region;

ALTER TABLE: modification du schéma

Action : Rename (Attribut)

ALTER TABLE nom_table ***RENAME*** **COLUMN** *ancien_nom TO nouveau_nom;*

- ALTER TABLE Film

  Rename Region TO Ville;

ALTER TABLE: modification du schéma

Action : Rename (Table)

ALTER TABLE *ancien_nomTable* ***RENAME*** *TO nouveau_nom;*

- ALTER TABLE Film

  Rename *TO* Films;

TRUNCATE

- Permet de vider une table et libérer l'espace de stockage utilisé par la table
- Syntaxe

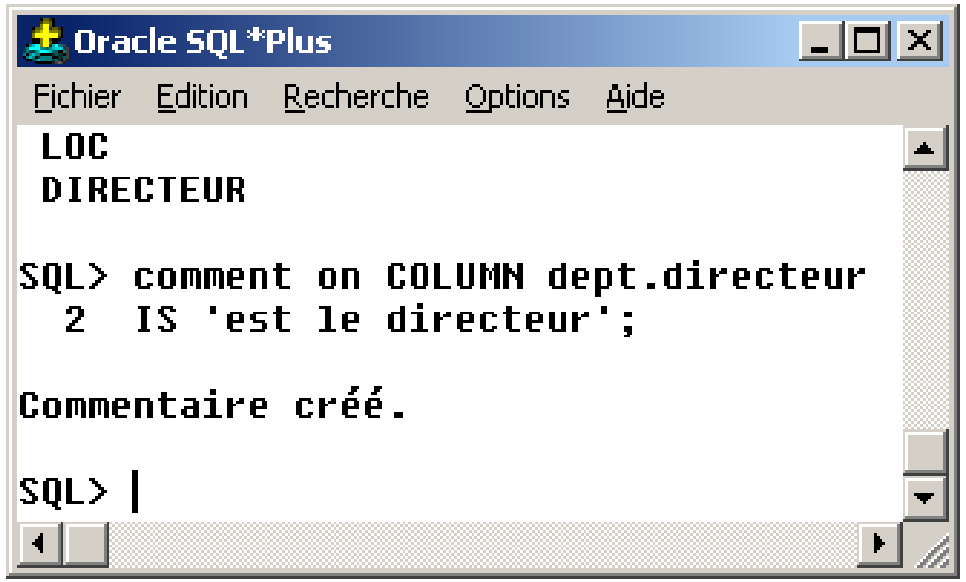
```
TRUNCATE TABLE nom_table;
```

- Vous pouvez utiliser DELETE pour supprimer les lignes d'une table
  - Mais il ne libère pas l'espace de stockage

COMMENT ON

- Permet d'ajouter des commentaires à une table
- Syntaxe

```
COMMENT ON [TABLE nom_table | COLUMN nom_table.nom_colonne]
         IS 'commentaire';
```



Insertion des données

INSERT INTO

- Permet d'ajouter de nouveaux enregistrements dans une table
- Syntaxe

```
INSERT INTO nom_table [(colonne_1 [, colonne_2…)]
       VALUES (Valeur_colonne_1 [, valeur_colonne_2…]
);
```

- Placez les valeurs de type caractère et date entre simples quotes

Insertion des données

INSERT INTO

- En précisant les colonnes concernées

```
INSERT INTO Emp (empno, ename, sal)
VALUES  (5555, 'BARTH', 2500);
```

- Sans précision

```
INSERT INTO Emp
VALUES (5555, 'BARTH', NULL, NULL, SYSDATE, 2500, NULL, NULL);
```

Insertion des données

INSERT INTO

```
CREATE TABLE Pays
        (code VARCHAR (4) DEFAULT 0 NOT NULL,
        nom VARCHAR (30) NOT NULL,
        langue VARCHAR (30) NOT NULL,
        PRIMARY KEY (code))
```

INSERT INTO Pays VALUES (0, ‘Inconnu’, ’Inconnue’);
INSERT INTO Pays VALUES (1, ’France’, ’Français’);
INSERT INTO Pays VALUES (2, ’USA’, ’Anglais’);
INSERT INTO Pays VALUES (3, ’Allemagne’, ’Allemand’);
INSERT INTO Pays VALUES (4, ’Angleterre’, ’Anglais’);

Insertion des données

INSERT INTO

- Avec des variables saisies par l'utilisateur

```
INSERT INTO Emp (empno, ename, sal)
      VALUES (&Num_Salarie, '&Nom_salarie', &salaire);
```

Insertion des données

INSERT INTO

- A partir d'une autre table

```
INSERT INTO Emp (empno, ename, sal)
          SELECT empno, ename, sal FROM SCOTT.EMP WHERE job = 'manager';
```

Modification des données

UPDATE

- Permet de modifier les enregistrements existants
  - A partir des valeurs

```
UPDATE Nom_table
       SET Colonne_1 = valeur_1 [, Colonne_2 = …]
       [WHERE condition] ;
```

  - A partir d'une requête

```
UPDATE Nom_table
        SET (Colonne_1, Colonne_2, ) =
        (SELECT colonne_11, Colonne_22, …
         FROM nom_table_2
         WHERE condition)
      [WHERE condition] ;
```

Modification des données

UPDATE

- Exemple
  - Augmenter de **10 %** le salaire de tous les employés appartenant au **département numéro 20**.

```
UPDATE Employe
SET salaire = salaire * 1.10
WHERE deptno = 20;
```

Suppression des données

DELETE

- Permet de supprimer les enregistrements d'une table

```
DELETE [FROM]nom_table
      [WHERE condition] ;
```

☠Attention, si vous omettez la clause WHERE, toutes les lignes sont supprimées

Au revoir et au prochain cours

Dr. Salma Sassi

salma.sassi@ece.fr